0057. Insert Interval

: Medium | Category: Intervals | LeetCode: https://leetcode.com/problems/insert-

Problem

- Insert newInterval into a sorted list of non-overlapping intervals, merging where needed.

Intuition

- Three phases: intervals entirely before the new one, intervals overlapping it (merge them in), intervals entirely after.

Brute Force

- Append, sort, then run Merge Intervals: O(n log n).

Optimal Approach

- Copy intervals with end < new.start.
- While interval.start <= new.end: new = [min of starts, max of ends].
- Add new, then copy the rest.

Worked Example

```
[[1,2],[3,5],[6,7],[8,10],[12,16]] + [4,8].

before: [1,2]
merge : [3,5] [6,7] [8,10] -> [3,10]
after : [12,16]
result [[1,2],[3,10],[12,16]]
```

Java Solution

```java
public class InsertInterval {
    public int[][] insert(int[][] intervals, int[] newInterval) {
        List<int[]> res = new ArrayList<>();
        int i = 0, n = intervals.length;
        int start = newInterval[0], end = newInterval[1];
        while (i < n && intervals[i][1] < start) res.add(intervals[i++]);
        while (i < n && intervals[i][0] <= end) {
            start = Math.min(start, intervals[i][0]);
            end = Math.max(end, intervals[i][1]);
            i++;
        }
        res.add(new int[]{start, end});
        while (i < n) res.add(intervals[i++]);
        return res.toArray(new int[0][]);
    }
}
```

Complexity

- Time: O(n). Space: O(n) for the output.

Pitfalls

- Touching intervals ([1,5] and [5,7]) overlap: use <=.